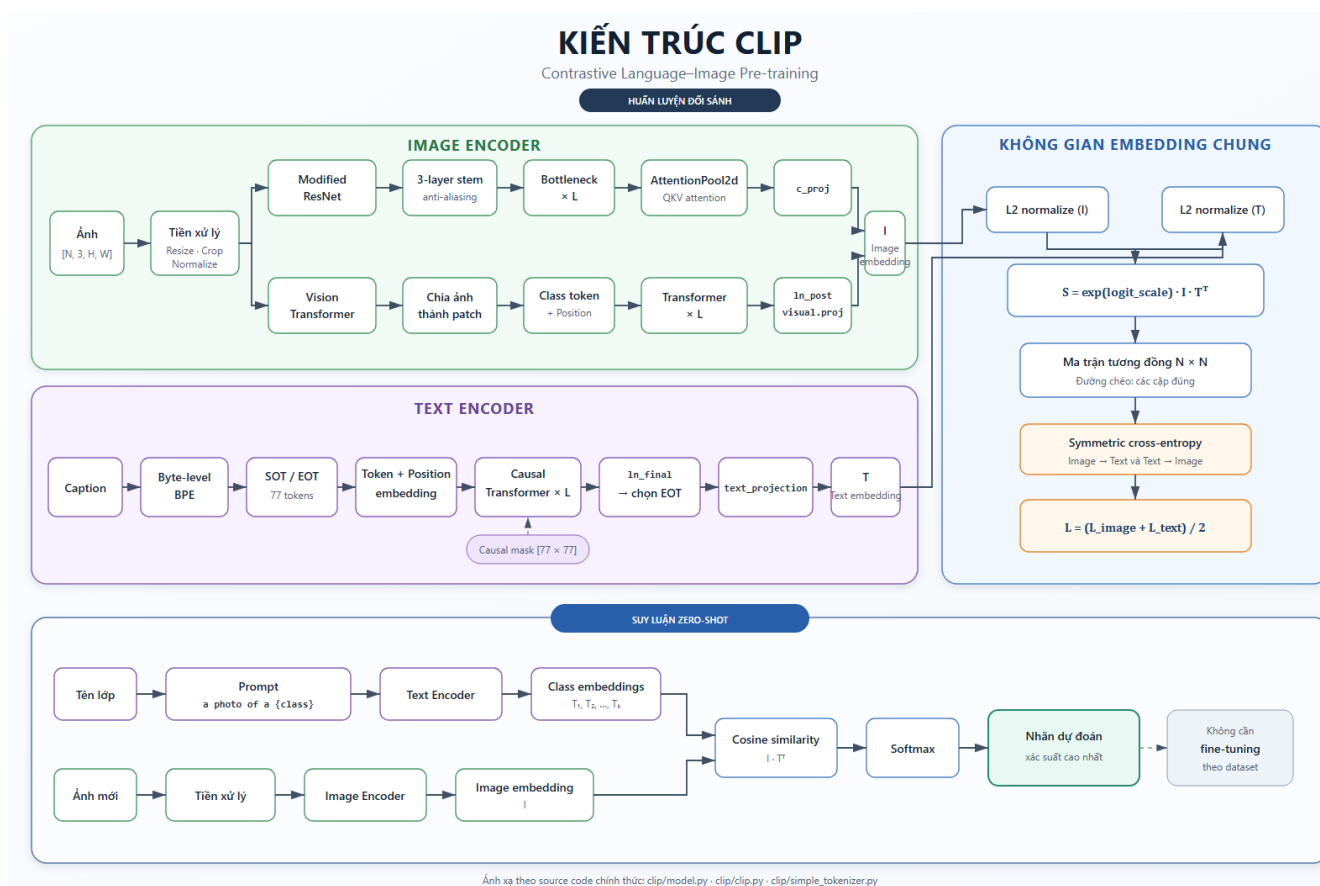


Câu 4 Giải thích source code và kiến trúc CLIP

Đổi chiếu kiến trúc với repository chính thức của OpenAI

CLIP là mô hình hai encoder. Image Encoder biến ảnh thành một vector, Text Encoder biến văn bản thành một vector có cùng số chiều, sau đó mô hình so sánh hai vector bằng cosine similarity. Điểm cốt lõi của kiến trúc là ảnh và văn bản không được ghép vào một Transformer chung; chúng được mã hóa độc lập rồi gặp nhau trong không gian embedding đa phương thức.

Phần trình bày dưới đây lần theo đúng luồng thực thi của repository openai/CLIP: tiền xử lý đầu vào, hai lựa chọn cho bộ mã hóa ảnh, Transformer văn bản, phép chiếu tuyến tính, chuẩn hóa L2, ma trận logits và cơ chế phân loại zero-shot. Source code phát hành chủ yếu phục vụ tải mô hình và suy luận; training loop cùng hàm loss đầy đủ được mô tả trong bài báo thay vì được đóng gói thành một hàm huấn luyện trong repository.



Hình 1. Pipeline tổng thể của kiến trúc CLIP và mối liên hệ giữa huấn luyện đối sánh với suy luận zero-shot.

1 Tổng quan kiến trúc

Với một mini-batch gồm N cặp ảnh và caption tương ứng, CLIP tạo N image embeddings và N text embeddings. Sau khi chuẩn hóa, tích ma trận giữa hai tập vector tạo ra ma trận tương đồng $N \times N$. Phần tử trên đường chéo biểu diễn cặp ảnh - văn bản đúng; các phần tử còn lại đóng vai trò cặp âm trong batch.

Lớp CLIP trong model.py là điểm nối của toàn bộ kiến trúc. Hàm khởi tạo tạo self.visual cho nhánh ảnh, self.transformer cho nhánh văn bản, text_projection cho phép chiếu văn bản và logit_scale cho nhiệt độ học được. Hai hàm encode_image và encode_text cung cấp embedding riêng, còn forward thực hiện chuẩn hóa và tính logits.

2 Tổ chức source code

Tệp	Thành phần chính	Vai trò
clip/clip.py	load, tokenize, _transform	API người dùng, tải checkpoint, tiền xử lý ảnh và đóng gói token văn bản.
clip/simple_tokenizer.py	SimpleTokenizer	Byte-level BPE, xây dựng từ điển token và chuyển văn bản thành token ID.
clip/model.py	ModifiedResNet, VisionTransformer, Transformer, CLIP	Định nghĩa toàn bộ mạng neural, phép chiếu embedding và phép tính logits.
README.md	Ví dụ zero-shot	Minh họa cách tạo prompt lớp, mã hóa ảnh và văn bản, sau đó chọn lớp theo similarity.
model-card.md	Mô tả mô hình	Tóm tắt mục đích, biến thể kiến trúc, giới hạn và phạm vi sử dụng.

3 Tiền xử lý dữ liệu

3 1 Tiền xử lý ảnh

Hàm _transform trong clip.py đưa ảnh về đúng định dạng mà checkpoint yêu cầu. Chuỗi thao tác gồm Resize bằng nội suy bicubic, CenterCrop, chuyển sang RGB, ToTensor và Normalize bằng mean cùng standard deviation của CLIP. Kích thước đích được lấy từ model.input_resolution, vì vậy cùng một API có thể phục vụ checkpoint 224 px hoặc 336 px.

```
Resize(n_px, interpolation=BICUBIC)
CenterCrop(n_px)
_convert_image_to_rgb
ToTensor()
Normalize(mean, std)
```

3 2 Tiền xử lý văn bản

SimpleTokenizer sử dụng byte-level BPE. Hàm clip.tokenize thêm token bắt đầu SOT và token kết thúc EOT, rồi đếm chuỗi thành context length 77. Nếu văn bản vượt giới hạn, người dùng phải cho phép

truncate; khi đó token cuối vẫn được thay bằng EOT. Repository chính thức không dùng SentencePiece.

4 Image Encoder

Lớp CLIP chọn bộ mã hóa ảnh dựa trên kiểu của `vision_layers`. Tuple hoặc list biểu thị bốn tầng ResNet; một số nguyên biểu thị số lớp Transformer của ViT. Vì vậy cùng lớp CLIP có thể dựng các checkpoint RN50, RN101, RN50x4, ViT-B/32, ViT-B/16 hoặc ViT-L/14.

```
if isinstance(vision_layers, (tuple, list)):
    self.visual = ModifiedResNet(...)
else:
    self.visual = VisionTransformer(...)
```

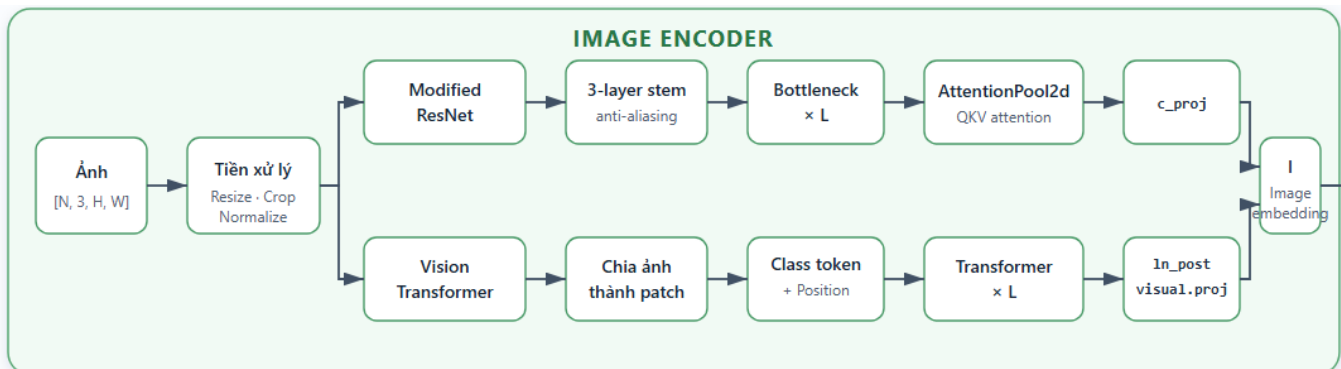
4 1 Modified ResNet

ModifiedResNet giữ các khối Bottleneck và residual connection của ResNet nhưng thay đổi ba điểm. Phần stem dùng ba convolution 3×3 ; quá trình giảm kích thước đặt average pooling trước convolution có stride; lớp pooling cuối được thay bằng AttentionPool2d. Đây là khác biệt quan trọng so với ResNet-50 thông thường, vì CLIP không kết thúc nhánh này bằng global average pooling.

AttentionPool2d trải feature map NCHW thành chuỗi HW vị trí, thêm một token trung bình toàn cục và positional embedding, sau đó dùng multi-head QKV attention. Kết quả được đưa qua `c_proj` để có kích thước `embed_dim`.

4 2 Vision Transformer

VisionTransformer dùng một Conv2d có `kernel_size` và `stride` bằng `patch_size` để chia ảnh thành patch. Các patch được làm phẳng thành chuỗi, ghép thêm `class_embedding`, cộng `positional_embedding` và chuẩn hóa bằng `ln_pre`. Sau các `ResidualAttentionBlock`, mô hình lấy token đầu tiên, áp dụng `ln_post` và nhân với `visual.proj`. Vector trả về đã nằm trong không gian embedding chung.



Hình 2. Hai phương án Image Encoder trong source code CLIP.

5 Text Encoder

Text Encoder là Transformer dùng masked self-attention. Mỗi ResidualAttentionBlock thực hiện hai phép cộng residual: attention trên đầu ra LayerNorm thứ nhất và MLP trên đầu ra LayerNorm thứ hai. MLP mở rộng chiều ẩn lên bốn lần, áp dụng QuickGELU rồi chiếu về chiều ban đầu.

```
x = x + self.attention(self.ln_1(x))
x = x + self.mlp(self.ln_2(x))
```

5.1 Causal attention mask

build_attention_mask tạo ma trận context_length × context_length, điền $-\infty$ phía trên đường chéo. Khi MultiheadAttention cộng mask vào attention scores, một token không thể nhìn các token đứng sau nó. Vision Transformer không nhận mask này, nên các patch ảnh vẫn attention toàn cục.

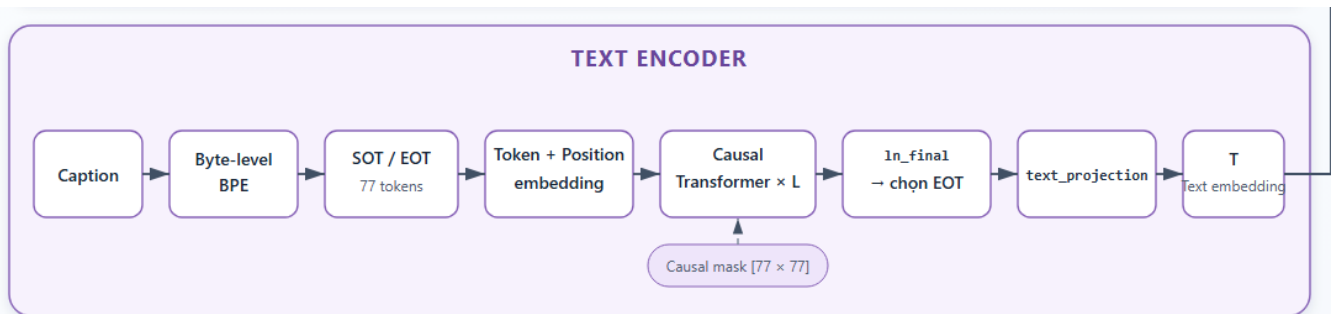
```
mask = torch.empty(self.context_length, self.context_length)
mask.fill_(float('-inf'))
mask.triu_(1)
```

5.2 Tạo text embedding

encode_text lần lượt tạo token embedding, cộng positional embedding, đổi thứ tự tensor sang LND để phù hợp với nn.MultiheadAttention và chạy qua Transformer. Sau khi đổi lại NLD, mô hình áp dụng ln_final, lấy hidden state tại token EOT và nhân với text_projection.

```
x = self.token_embedding(text).type(self.dtype)
x = x + self.positional_embedding.type(self.dtype)
x = self.transformer(x.permute(1, 0, 2)).permute(1, 0, 2)
x = self.ln_final(x).type(self.dtype)
x = x[torch.arange(x.shape[0]), text.argmax(dim=-1)] @ self.text_projection
```

Cách dùng text.argmax(dim=-1) hoạt động vì EOT có token ID lớn nhất trong từ điển. Sau phép nhân text_projection, đầu ra có dạng [batch_size, embed_dim], cùng chiều với image embedding.



Hình 3. Thứ tự xử lý chính xác trong Text Encoder.

6 Không gian embedding chung

encode_image và encode_text đều trả về vector đã được chiếu sang embed_dim. Trong forward, hai tập vector được chuẩn hóa L2. Khi đó tích vô hướng giữa hai vector chuẩn hóa chính là cosine similarity.

$$\hat{I} = I / \|I\|_2 \quad \hat{T} = T / \|T\|_2$$

logit_scale được lưu ở dạng log và được lấy exp trước khi nhân với ma trận similarity. Cách tham số hóa này giữ hệ số scale dương. Giá trị khởi tạo tương ứng với temperature 0,07.

$$S = \exp(s) \cdot \hat{I} \cdot \hat{T}^T$$

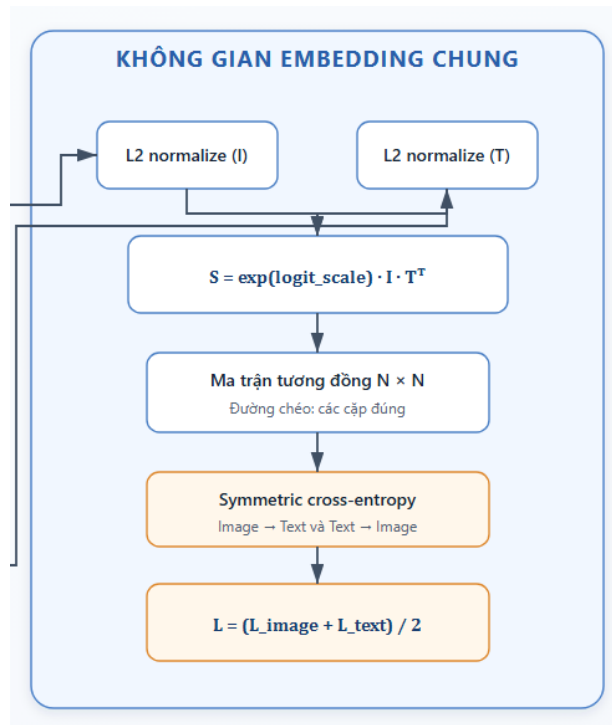
```
image_features = image_features / image_features.norm(dim=1, keepdim=True)
text_features = text_features / text_features.norm(dim=1, keepdim=True)
logit_scale = self.logit_scale.exp()
logits_per_image = logit_scale * image_features @ text_features.t()
logits_per_text = logits_per_image.t()
```

7 Mục tiêu huấn luyện đối sánh

Với batch gồm N cặp đúng, ma trận S có kích thước $N \times N$. Nhãn mục tiêu là các chỉ số $[0, 1, \dots, N - 1]$, nên cặp đúng của hàng i nằm ở cột i. Cross-entropy thứ nhất yêu cầu mỗi ảnh chọn đúng caption; cross-entropy thứ hai yêu cầu mỗi caption chọn đúng ảnh. Loss cuối là trung bình của hai hướng.

$$L = \frac{1}{2} [CE(S, y) + CE(S^T, y)] \quad y = (0, 1, \dots, N - 1)$$

Repository openai/CLIP không định nghĩa training loop hoặc hàm contrastive loss hoàn chỉnh. Hàm forward chỉ trả logits_per_image và logits_per_text; công thức loss đối xứng xuất hiện trong bài báo. Vì vậy phần loss trong sơ đồ là thành phần của phương pháp huấn luyện, không phải một lớp có sẵn trong model.py.



Hình 4. Chuẩn hóa embedding, ma trận tương đồng và contrastive loss đối xứng.

8 Suy luận zero shot

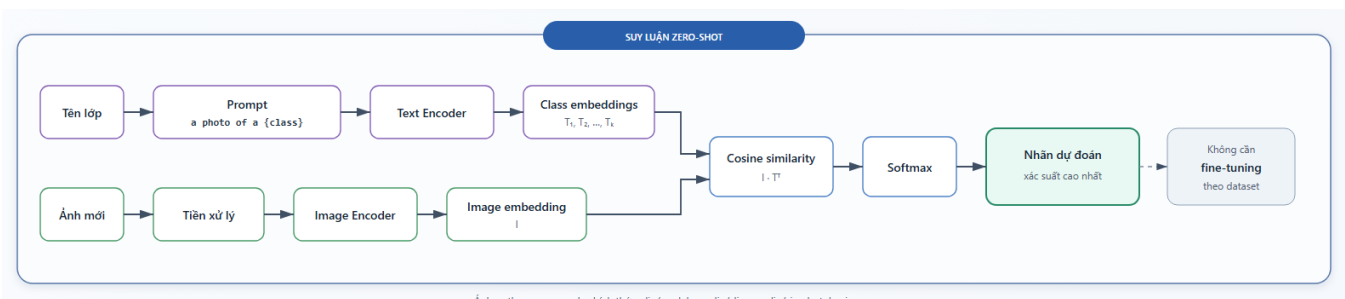
CLIP thực hiện phân loại zero-shot bằng cách biến tên lớp thành văn bản mô tả. Ví dụ, lớp dog được đặt vào prompt a photo of a dog. Text Encoder mã hóa từng prompt thành một class embedding; tập embedding này đóng vai trò bộ phân loại tuyến tính được tạo động từ ngôn ngữ.

Ảnh cần dự đoán được tiền xử lý và đưa vào Image Encoder. Sau khi chuẩn hóa, image embedding được nhân với ma trận class embeddings. Softmax theo chiều lớp biến các logits thành phân phối xác suất, và lớp có xác suất cao nhất được chọn.

```
text_inputs = torch.cat([
    clip.tokenize(f'a photo of a {class_name}')
    for class_name in classes
])

image_features = model.encode_image(image_input)
text_features = model.encode_text(text_inputs)
image_features /= image_features.norm(dim=-1, keepdim=True)
text_features /= text_features.norm(dim=-1, keepdim=True)
probs = (100.0 * image_features @ text_features.T).softmax(dim=-1)
```

Trong luồng này, tên lớp và prompt luôn đi vào Text Encoder, còn ảnh mới luôn đi vào Image Encoder. Mô hình không cần thêm classification head cố định và cũng không cần fine-tuning nếu chỉ thực hiện zero-shot inference.



Hình 5. Pipeline phân loại zero-shot của CLIP.

9 Tải checkpoint và khôi phục kiến trúc

Hàm clip.load tải checkpoint theo tên mô hình hoặc nhận đường dẫn cục bộ. Khi dùng state_dict, build_model suy ra cấu hình kiến trúc từ hình dạng của trọng số. Nếu state_dict chứa visual.proj thì checkpoint dùng ViT; nếu không, hàm đếm số Bottleneck trong visual.layer1 đến visual.layer4 để dựng Modified ResNet. Cách này giúp repository không phải hard-code toàn bộ tham số của từng biến thể.

Sau khi dựng mô hình, convert_weights chuyển các convolution, linear, projection và tham số attention thích hợp sang FP16. State dictionary được nạp và mô hình được trả về ở chế độ eval. clip.load đồng thời trả preprocess tương ứng với input_resolution của checkpoint.

10 Đối chiếu kiến trúc với source code

Thành phần kiến trúc	Lớp hoặc hàm	Kết quả
Tiền xử lý ảnh	clip.py::_transform	Tensor ảnh [N, 3, H, W] đã resize, crop và normalize.
Tokenization	SimpleTokenizer; clip.tokenize	Tensor token [N, 77] có SOT và EOT.
ResNet image encoder	Bottleneck; ModifiedResNet; AttentionPool2d	Image embedding sau c_proj.
ViT image encoder	VisionTransformer	Image embedding lấy từ class token sau visual.proj.
Khối Transformer	ResidualAttentionBlock; Transformer	Chuỗi đặc trưng sau self-attention và MLP.
Text encoder	CLIP.encode_text	Vector EOT sau ln_final và text_projection.
Embedding ảnh	CLIP.encode_image	Đầu ra self.visual có kích thước embed_dim.
Similarity logits	CLIP.forward	logits_per_image và logits_per_text.
Khôi phục mô hình	build_model	Kiến trúc được suy ra từ state_dict.
Zero-shot	README.md example	Xác suất trên các lớp được tạo từ prompt văn bản.

11 Kết luận

Source code phản ánh trực tiếp thiết kế dual encoder của CLIP. Nhánh ảnh dùng Modified ResNet có AttentionPool2d hoặc Vision Transformer; nhánh văn bản dùng causal Transformer và lấy biểu diễn tại EOT. Hai đầu ra được chiếu sang cùng không gian, chuẩn hóa và so sánh bằng cosine similarity có scale học được. Cùng cơ chế này hỗ trợ contrastive pre-training và cho phép xây dựng bộ phân loại zero-shot từ tên lớp bằng ngôn ngữ tự nhiên.

Tài liệu tham khảo

1. OpenAI CLIP repository. <https://github.com/openai/CLIP>
2. Source code model.py. <https://github.com/openai/CLIP/blob/main/clip/model.py>
3. Source code clip.py. <https://github.com/openai/CLIP/blob/main/clip/clip.py>
4. Radford và cộng sự Learning Transferable Visual Models From Natural Language Supervision. <https://arxiv.org/abs/2103.00020>